

Conditioning and Directionality in Adversarial Transfer on CIFAR-10

Jaiveer Bassi
Independent Researcher
jaiveerbassi@yahoo.com

September 2026

Abstract

Conventionally reported source attack success rate (ASR) and pairwise transfer rate (PTR) cannot reconstruct transfer conditional on source success (CTR): source ASR uses a broader denominator than the pair-dependent population required by CTR, and it omits target behavior when the source attack fails. We demonstrate this distinction on the full CIFAR-10 test set using three independently trained ResNet-18, VGG16, and MobileNetV2 checkpoints. The corrected evaluation excludes existing clean errors and reports exact denominators. At 2/255, conditioning is nearly mechanical in five of six directions: target error among eligible source failures is at most 0.68%, and CTR is within 0.36 points of the zero-residual value PTR/a_{st} . VGG16-to-MobileNetV2 is the exception, with a 2.83% failed-source residual and a 1.17-point difference. At 8/255, mean PGD PTR ranges from 18.09% for ResNet-to-VGG to 95.23% for VGG-to-MobileNet. A seed-0 control training all architectures at learning rate 0.01 preserves the VGG transfer asymmetry: its outbound PTR is 93.12–95.98%, versus 18.55–32.71% in reverse. The result is a seed-replicated, denominator-complete account of when source conditioning adds information and when it acts mainly as arithmetic rescaling.

1 Introduction

Adversarial examples expose a gap between ordinary predictive accuracy and behavior under deliberately chosen perturbations. FGSM provides a gradient-based construction [1], while iterative optimization enables broader searches within a threat model [2, 3]. Transfer attacks use a source model to construct an input that is subsequently evaluated on another model. The target supplies neither gradients nor feedback during construction.

Transferability is an empirical property of a source, a target, an attack, and an evaluation population. It cannot be inferred from a high error rate alone. If the target already misclassifies most clean inputs, an unchanged input may appear to be a successful attack under an unconditional metric. A mismatch between label spaces makes this problem more severe.

The study asks whether conclusions about attack rank-

ings and model-pair differences are stable under explicitly stated conditioning populations. Its contributions are:

1. an exact decomposition showing why conventional source ASR and PTR do not determine CTR;
2. a full-test, three-seed measurement with jointly clean populations, exact counts, and PGD PTR spanning 18.09–95.23% at 8/255; and
3. convergence and matched-learning-rate controls that preserve the main directional result while exposing its dependence on optimization.

The self-correction in Section 3 motivates the measurement question and is not evidence of a general failure rate across the literature.

2 Background and scope

Goodfellow et al. connect adversarial vulnerability to high-dimensional linear behavior and introduce FGSM [1]. Madry et al. formulate robustness using optimization over a perturbation set [2]. Carlini and Wagner develop optimization-based attacks and demonstrate the importance of careful evaluation [3]. Papernot et al. demonstrate practical transfer-based black-box attacks using substitute models [4]. Liu et al. study non-targeted and targeted transfer across large models [5], while Tramer et al. characterize shared adversarial subspaces and limits of transfer [6].

Directionality and the tension between white-box optimization and transfer are established phenomena. Demontis et al. explain asymmetric transfer through properties including target vulnerability and surrogate complexity [8]. Dong et al. introduce momentum to stabilize iterative attack directions and improve black-box transfer [7]. Our contribution is not the rediscovery of asymmetry or source overfitting. We instead provide a seed-replicated remeasurement with explicit jointly clean populations, source-success strata, and exact counts, and quantify when CTR is almost determined by source success and when a failed-source residual remains.

Carlini et al. systematize the methodological foundations and common pitfalls of adversarial-robustness evaluation [10]. Croce and Hein introduce AutoAttack, an

ensemble intended to reduce dependence on attack tuning and improve evaluation reliability [11]. Tramer et al. emphasize defense-specific adaptive evaluation and illustrate how apparently adequate evaluations can miss weaknesses [12]. RobustBench standardizes robustness benchmarking under specified tasks and threat models and uses AutoAttack as part of its evaluation framework [13].

Our study complements these evaluation efforts by examining source-to-target transfer and the populations on which transfer rates are conditioned. It does not introduce a stronger attack, certify robustness, or replace adaptive defense evaluation. The FGSM/PGD study does not constitute an AutoAttack evaluation; citing AutoAttack and RobustBench must not be interpreted as having run either benchmark. Attack-strength sensitivity and the source white-box controls are necessary for interpreting the transfer measurements.

The present protocol concerns untargeted digital attacks against deterministic image classifiers. It does not evaluate physical attacks, detection systems, adversarially trained defenses, or deployment security. CIFAR-10 results would not by themselves justify claims about autonomous vehicles or face recognition. The selected architectures are established CNN baselines, not a comprehensive representation of contemporary vision systems.

3 Motivating case study: a self-correction

Repository commit 58f8102 compared CIFAR-10 labels directly with the indices of 1,000-class ImageNet models, omitted required preprocessing, mixed perturbation coordinate systems, and counted existing clean errors as attack successes. It also lacked compatible checkpoints and per-example logs. Those rates and rankings are excluded from the revised empirical record. This self-correction motivates compatible labels, raw-pixel budgets, clean eligibility, and recorded source success; it is not the empirical contribution. The line-by-line audit is retained in the supplementary artifact.

4 Why error is not attack success

Let f_t be the target predictor, y the true label, and $A_s(x)$ an attack constructed using source s . Define clean error $e_t = \Pr[f_t(x) \neq y]$, newly induced error $q = \Pr[f_t(A_s(x)) \neq y \mid f_t(x) = y]$, and persistence of existing error $r = \Pr[f_t(A_s(x)) \neq y \mid f_t(x) \neq y]$. By partitioning on clean correctness,

$$\Pr[f_t(A_s(x)) \neq y] = (1 - e_t)q + e_tr. \quad (1)$$

For the identity attack, $q = 0$ and $r = 1$, so adversarial error equals clean error. An illustrative classifier with 99% clean error therefore has 99% adversarial error under

an identity attack and zero newly induced errors. This is an algebraic example, not a measurement from CIFAR-10.

The unconditional attacked-error rate and CTR condition on different populations, so their difference is not an estimate of inflation caused by clean errors. Equation (1) attributes all-input attacked error to newly induced and persistent clean errors; PTR and CTR then add source-correctness and source-success conditions, with empty conditioning populations remaining undefined.

4.1 Recorded estimands

For a fixed evaluation set of N examples, let $C_s(i)$ and $C_t(i)$ indicate clean correctness. Let $S_s(i)$ and $S_t(i)$ indicate misclassification of the source-generated attacked input. Define the pairwise eligible set $E_{st} = \{i : C_s(i) \wedge C_t(i)\}$ and successful-source subset $F_{st} = \{i \in E_{st} : S_s(i)\}$. We report

$$\text{ASR}_s = \frac{\sum_{i:C_s(i)} S_s(i)}{\sum_i C_s(i)}, \quad (2)$$

$$\text{PTR}_{st} = \frac{\sum_{i \in E_{st}} S_t(i)}{|E_{st}|}, \quad (3)$$

$$\text{CTR}_{st} = \frac{\sum_{i \in F_{st}} S_t(i)}{|F_{st}|}. \quad (4)$$

PTR measures target failure on jointly clean-correct inputs whether or not the source attack succeeded. CTR measures transfer conditional on source success. To expose the arithmetic effect of conditioning, define pairwise source success $a_{st} = \Pr[S_s \mid E_{st}]$ and failed-source target error $b_{st} = \Pr[S_t \mid E_{st}, \neg S_s]$. Partitioning the eligible set gives the exact identity

$$\text{PTR}_{st} = a_{st} \text{CTR}_{st} + (1 - a_{st})b_{st}. \quad (5)$$

Consequently, $\text{CTR}_{st} = \text{PTR}_{st}/a_{st}$ only when $b_{st} = 0$. The conventional ASR_s above cannot be substituted for a_{st} because its denominator includes source-correct inputs on which the target was already wrong. We report a_{st} and b_{st} for the conditioning analysis. The diagonal is a white-box control; its CTR is tautologically one whenever its denominator is positive.

All fractions include numerator and denominator counts. An empty denominator is undefined, not zero. Clean accuracy and attacked accuracy on the full evaluation set are also reported. Wilson 95% intervals describe uncertainty under a binomial sampling interpretation for fixed checkpoints; they are descriptive when evaluating the entire finite CIFAR-10 test set and do not measure variability across training runs. Pairwise eligibility sets differ, so raw rates across model pairs are not strictly matched-population comparisons.

5 Experimental protocol

5.1 Data and model training

The pipeline uses the standard CIFAR-10 class order [9]. A fixed split seed of 1729 partitions the 50,000 training images into 45,000 training and 5,000 validation images. Validation data receive no random augmentation. Training uses random crops with four-pixel padding and random horizontal flips. The 10,000 test images are reserved for final evaluation; checkpoint selection uses validation accuracy only.

All networks have ten output classes and are initialized without ImageNet weights. The ResNet-18 stem uses a 3×3 convolution at stride one and no initial max pool. VGG16 uses its convolutional feature stack, global 1×1 pooling, and a single 512-to-10 linear classifier. MobileNetV2 changes its initial convolution to stride one. These adaptations, especially the compact VGG classifier, must be stated when comparing with other implementations.

Inputs to the attack interface are floating-point pixels in $[0, 1]$. Channel normalization with mean (0.4914, 0.4822, 0.4465) and standard deviation (0.2470, 0.2435, 0.2616) occurs inside the model, so gradients include preprocessing while perturbation budgets remain in raw-pixel coordinates. Checkpoints contain architecture, class order, split indices, training seed, selected epoch, and model state. Evaluation rejects incompatible checkpoint metadata.

Training uses 200 epochs, SGD with momentum 0.9, weight decay 5×10^{-4} , cosine learning-rate decay, and batch size 128. ResNet-18 and MobileNetV2 use initial learning rate 0.1. That rate caused the VGG16 loss to remain near $\log 10$ and validation accuracy near chance, so we stopped that failed run and selected 0.01 using validation performance only. We then froze 0.01 for all three VGG16 seeds before test evaluation. The independently trained seeds are 0, 1, and 2; the split seed remains 1729.

5.2 Attacks and controls

FGSM uses one signed cross-entropy gradient step with projection into the pixel range. PGD uses random initialization within the intersection of the pixel range and the L_∞ ball, 40 steps, absolute step size $2/255$, and five restarts at the primary budget $8/255$. Across iterates and restarts, the implementation prioritizes source misclassification, then larger source loss. Target predictions never influence this selection. This source-selected construction may differ from final-iterate PGD in its transfer behavior, so the selection rule is part of the attack specification.

The clean control leaves inputs unchanged. The random-noise control samples independent uniform perturbations within the same L_∞ budget and clips to the pixel range. The empirical scope is limited to the L_∞ threat model. We do not mix CW- L_2 results with these

measurements or imply that equal numeric radii across norms represent equal threat models.

For every source and attack, the same generated inputs are evaluated on every target, including the source. Models remain in evaluation mode. Runtime checks reject nonfinite pixels, invalid pixel ranges, and norm violations. A saved example records its test index, true label, predictions, and actual perturbation. Visualization does not presume attack success or establish perceptual invisibility.

5.3 Reproducibility and sensitivity analysis

A seeded permutation specifies test indices, and the identical ordered sample list is used for every pair. The final study should use the full test set. A 1,000-example subset is suitable for a clearly labeled pilot only. Each completed run saves arguments, software versions, commit identifier, checkpoint SHA-256 hashes, test indices, elapsed time, clean and attacked per-example predictions, and per-example L_2 and L_∞ norms. Tables are generated from measured summaries rather than embedded constants.

We evaluate L_∞ radii $2/255$, $4/255$, and $8/255$. The PGD step size is one quarter of the radius. At $8/255$, seed 0 uses 10, 40, and 100 steps with one restart and 100 steps with five restarts; seeds 1 and 2 repeat the 10-step, one-restart and 100-step, five-restart endpoints. At $2/255$, seed 0 also uses 100 steps and five restarts. Attack hyperparameters are not selected to maximize target transfer. We show training seeds individually and summarize seed-level estimates; we do not pool repeated test images as independent observations.

To test whether the architecture-specific VGG16 learning rate alone explains the largest asymmetries, we train seed-0 ResNet-18 and MobileNetV2 controls at 0.01 for the same 200 epochs and rerun the full-test $8/255$ protocol with the existing VGG16 seed-0 checkpoint. This is a prespecified sensitivity comparison; it is not pooled with the three-seed primary estimates.

6 Results

6.1 Clean accuracy and primary attack strength

Table 1 reports validation-selected checkpoint performance on the untouched test set. Variation across the three training seeds is small relative to the differences among architectures. At the primary $8/255$ budget, 40-step, five-restart PGD attains 100% source ASR for every ResNet-18 and MobileNetV2 checkpoint and at least 99.98% for every VGG16 checkpoint. The diagonal therefore confirms that the primary attack is strong against its source models.

Table 1: Clean test accuracy (%) by training seed. The final column is the mean and sample standard deviation over three checkpoints.

Model	Seed 0	Seed 1	Seed 2	Mean $\pm$ SD
ResNet-18	95.11	95.01	95.20	95.11 $\pm$ 0.10
VGG16	90.59	90.41	90.88	90.63 $\pm$ 0.24
MobileNetV2	89.97	90.57	90.38	90.31 $\pm$ 0.31

Table 2: Mean source ASR (%) over three seeds.

Source	FGSM 8/255	PGD 2/255	PGD 4/255	PGD 8/255
ResNet-18	58.54	94.47	99.91	100.00
VGG16	87.31	71.73	97.93	99.98
MobileNetV2	92.03	99.85	100.00	100.00

Uniform random noise at the same 8/255 radius produces mean off-diagonal PTR between 1.45% and 7.89%, below the corresponding adversarial results. Noise CTR is based on much smaller source-success subsets and is not used to rank model pairs. Because each nominal source row uses a separately seeded noise realization, these controls establish a non-adversarial baseline within each row rather than a paired comparison among noise rows.

6.2 Directionality at the primary budget

Table 3 shows off-diagonal PGD PTR at 8/255. Transfer is strongly directional. VGG16 examples transfer to ResNet-18 at 92.73% on average, whereas the reverse direction reaches 18.09%. MobileNetV2-to-VGG16 is 25.94%, whereas VGG16-to-MobileNetV2 is 95.23%. The ordering persists across all three seeds, so it is not produced by one checkpoint draw. These are associations among the trained architectures and recipes; differences in capacity, optimization, and clean accuracy preclude an architecture-only causal interpretation.

The matched-learning-rate control preserves the two largest VGG asymmetries (Table 4). VGG16-to-ResNet-18 exceeds the reverse direction by 63.27 points, and VGG16-to-MobileNetV2 exceeds the reverse by 74.57 points. The control ResNet-18 has 93.82% clean test accuracy, compared with 95.11% for its primary seed-0 checkpoint; control MobileNetV2 has 90.90%, compared with 89.97%. Individual transfer entries change by up to 25.93 points across the full matrix. The control rejects a learning-rate-only account of the VGG asymmetry; it licenses no stronger causal claim because the resulting checkpoints also differ in accuracy, optimization trajectory, and loss geometry.

6.3 Budget and conditioning sensitivity

Transfer falls sharply with the perturbation budget, but the directional ordering is broadly stable (Table 6). VGG16-to-ResNet-18 drops from 92.73% at 8/255 to 15.45% at 2/255; ResNet-18-to-MobileNetV2 drops from

Table 3: PGD PTR (%) at 8/255. Entries are the three-seed mean [envelope of seed-specific Wilson 95% intervals]. This interval envelope is descriptive and does not pool repeated test images.

Source	ResNet-18	VGG16	MobileNetV2
ResNet-18	—	18.09 [15.97,20.22]	72.27 [70.81,73.64]
VGG16	92.73 [91.83,93.50]	—	95.23 [94.31,96.02]
MobileNetV2	67.60 [62.45,71.55]	25.94 [24.81,27.23]	—

Table 4: Seed-0 PGD PTR (%) at 8/255 before and after training all three architectures with initial learning rate 0.01.

Direction	Primary	Matched LR
ResNet-18 $\rightarrow$ VGG16	19.38	32.71
VGG16 $\rightarrow$ ResNet-18	92.99	95.98
MobileNetV2 $\rightarrow$ VGG16	25.73	18.55
VGG16 $\rightarrow$ MobileNetV2	94.80	93.12

72.27% to 21.61%. These changes rule out treating one reported transfer percentage as a property of a model pair independent of its threat model.

At 2/255, VGG16 source ASR is only 71.05–72.98% (Table 2). Table 7 applies the exact conditioning identity with absolute counts. Conditioning is nearly mechanical in five directions: b_{st} is at most 0.68%, and observed CTR is within 0.36 points of PTR/a_{st} . VGG16-to-MobileNetV2 is the exception, with mean $b_{st} = 2.83\%$ and observed CTR 1.17 points below the zero-residual value. This exception confirms that the successful- and failed-source strata are not interchangeable. More fundamentally, conventional source ASR and PTR cannot determine CTR even when the residual is negligible, because conventional ASR uses a broader, target-independent denominator than a_{st} .

The stronger 100-step, five-restart 8/255 endpoint changes off-diagonal PTR by at most 1.70 percentage points relative to the primary 40-step, five-restart setting across all seeds. Increasing optimization does not increase every transfer entry: for example, ResNet-18-to-VGG16 falls slightly. At 2/255 on seed 0, increasing from 40 to 100 steps at five restarts changes all off-diagonal PTR entries by at most 0.27 points and CTR entries by at most 0.40 points. This supports convergence of the primary step count without asserting a global optimum.

FGSM reinforces the distinction between source strength and transfer (Table 5). At 8/255, its mean PTR is 27.80% for ResNet-18-to-VGG16 and 29.34% for MobileNetV2-to-VGG16, compared with 18.09% and 25.94% for PGD. PGD nevertheless has far higher source ASR. For the other four off-diagonal directions, PGD transfers more strongly than FGSM. This replicates prior observations that iterative source optimization can reduce transfer [7]; an attack ranking depends on direction and cannot be inferred from white-box source success alone.

Table 5: FGSM PTR (%) at 8/255, reported as mean [minimum, maximum] over three seeds.

Source	ResNet-18	VGG16	MobileNetV2
ResNet-18	—	27.80 [26.21,30.01]	58.30 [56.73,59.43]
VGG16	55.27 [54.14,56.75]	—	64.88 [64.38,65.71]
MobileNetV2	47.37 [47.20,47.55]	29.34 [28.09,30.55]	—

Table 6: Mean PGD PTR (%) over three seeds as a function of L_∞ budget.

Direction	2/255	4/255	8/255
ResNet-18 $\rightarrow$ VGG16	5.84	9.65	18.09
ResNet-18 $\rightarrow$ MobileNetV2	21.61	42.40	72.27
VGG16 $\rightarrow$ ResNet-18	15.45	54.52	92.73
VGG16 $\rightarrow$ MobileNetV2	23.29	64.50	95.23
MobileNetV2 $\rightarrow$ ResNet-18	10.79	31.80	67.60
MobileNetV2 $\rightarrow$ VGG16	4.61	10.69	25.94

7 Evidence status and limitations

The code, manuscript, and reproducibility snapshot are archived under concept DOI 10.5281/zenodo.22737839; the archived v1.0.0 study snapshot is 10.5281/zenodo.22737840. The final record contains nine checkpoints, full-test predictions, exact eligibility counts, perturbation norms, manifests, and checkpoint and source hashes. The three seed triads provide evidence about variability under the specified training procedure, but three draws support descriptive ranges rather than precise population-level inference. Wilson intervals in the generated per-run tables describe fixed-checkpoint binomial variation and do not capture training variability. The same 10,000 test images recur across seeds, so image-level counts must not be pooled as 30,000 independent observations.

The study covers three standard CNN families on CIFAR-10 and untargeted L_∞ attacks. It does not establish behavior on larger datasets, transformers, adversarially trained models, targeted attacks, or physical settings. It also does not compare against every modern transfer attack. The VGG16 learning-rate adjustment was selected using validation results after the common recipe failed. A seed-0 matched-learning-rate control preserves its strongest asymmetries, but only one control seed was run and some individual entries change substantially; optimization and architecture remain entangled. Random-noise draws are independently generated for each nominal source row, so noise-control rows are reproducible but do not share one common perturbation realization. The retained manifests preserve prediction checksums, checkpoint hashes, and hashes of the executed source files.

The original artifact is a motivating self-correction, not evidence that similar errors are common elsewhere. The present results do not imply that one architecture is intrinsically more transferable: architecture, parameterization, training dynamics, and clean accuracy vary together. PTR and CTR answer different questions, and

Table 7: 2/255 PGD conditioning decomposition. Count tuples list seeds 0/1/2; rates are mean [range] percentages.

Direction	Pairwise eligible $ E $	Source success $ F $
R $\rightarrow$ V	8905/8874/8923	8381/8318/8424
R $\rightarrow$ M	8831/8894/8873	8307/8339/8374
V $\rightarrow$ R	8905/8874/8923	6457/6257/6303
V $\rightarrow$ M	8583/8589/8620	6139/5976/6003
M $\rightarrow$ R	8831/8894/8873	8822/8879/8857
M $\rightarrow$ V	8583/8589/8620	8574/8574/8604

Direction	Pair ASR	Fail-target	PTR	CTR
R $\rightarrow$ V	94.09 [93.73,94.41]	0.00 [0.00,0.00]	5.84	6.21
R $\rightarrow$ M	94.07 [93.76,94.38]	0.25 [0.19,0.36]	21.61	22.96
V $\rightarrow$ R	71.22 [70.51,72.51]	0.68 [0.42,0.96]	15.45	21.42
V $\rightarrow$ M	70.25 [69.58,71.53]	2.83 [2.71,2.99]	23.29	31.97
M $\rightarrow$ R	99.85 [99.82,99.90]	0.00 [0.00,0.00]	10.79	10.81
M $\rightarrow$ V	99.84 [99.81,99.90]	0.00 [0.00,0.00]	4.61	4.62

neither should be selected after seeing which produces the more dramatic result.

8 Conclusion

Across three training seeds, adversarial transfer on CIFAR-10 is strongly directional and budget dependent. At 8/255, mean PGD PTR spans 18.09–95.23% across off-diagonal pairs; at 2/255, it spans 4.61–23.29%. Conditioning is nearly mechanical in five directions, with one measurable failed-source exception. Yet conventional source ASR and PTR still cannot reconstruct CTR because source ASR uses the wrong, target-independent denominator. The strongest VGG directionality persists in a seed-0 matched-learning-rate control, while the movement of individual entries cautions against an architecture-only interpretation. Stronger source optimization can also reduce transfer for a particular direction, replicating established source-overfitting behavior. Compatible labels and preprocessing make these measurements meaningful; clean-error exclusions, explicit denominators, seed-level reporting, and convergence checks make their limits visible.

References

- [1] I. J. Goodfellow, J. Shlens, and C. Szegedy. Explaining and Harnessing Adversarial Examples. ICLR, 2015. <https://arxiv.org/abs/1412.6572>.
- [2] A. Madry, A. Makelov, L. Schmidt, D. Tsipras, and A. Vladu. Towards Deep Learning Models Resistant to Adversarial Attacks. ICLR, 2018. <https://arxiv.org/abs/1706.06083>.
- [3] N. Carlini and D. Wagner. Towards Evaluating the Robustness of Neural Networks. IEEE Symposium

- on Security and Privacy, 2017. <https://arxiv.org/abs/1608.04644>.
- [4] N. Papernot, P. McDaniel, I. Goodfellow, S. Jha, Z. B. Celik, and A. Swami. Practical Black-Box Attacks against Machine Learning. ACM AsiaCCS, 2017. <https://arxiv.org/abs/1602.02697>.
 - [5] Y. Liu, X. Chen, C. Liu, and D. Song. Delving into Transferable Adversarial Examples and Black-box Attacks. ICLR, 2017. <https://arxiv.org/abs/1611.02770>.
 - [6] F. Tramer, N. Papernot, I. Goodfellow, D. Boneh, and P. McDaniel. The Space of Transferable Adversarial Examples. arXiv:1704.03453, 2017. <https://arxiv.org/abs/1704.03453>.
 - [7] Y. Dong, F. Liao, T. Pang, H. Su, J. Zhu, X. Hu, and J. Li. Boosting Adversarial Attacks with Momentum. CVPR, 2018. https://openaccess.thecvf.com/content_cvpr_2018/html/Dong_Boosting_Adversarial_Attacks_CVPR_2018_paper.html.
 - [8] A. Demontis, M. Melis, M. Pintor, M. Jagielski, B. Biggio, A. Oprea, C. Nita-Rotaru, and F. Roli. Why Do Adversarial Attacks Transfer? Explaining Transferability of Evasion and Poisoning Attacks. USENIX Security, 2019. <https://www.usenix.org/conference/usenixsecurity19/presentation/demontis>.
 - [9] A. Krizhevsky. Learning Multiple Layers of Features from Tiny Images. Technical report, University of Toronto, 2009. <https://www.cs.toronto.edu/~kriz/cifar.html>.
 - [10] N. Carlini et al. On Evaluating Adversarial Robustness. arXiv:1902.06705, 2019. <https://arxiv.org/abs/1902.06705>.
 - [11] F. Croce and M. Hein. Reliable Evaluation of Adversarial Robustness with an Ensemble of Diverse Parameter-free Attacks. ICML, PMLR 119:2206–2216, 2020. <https://proceedings.mlr.press/v119/croce20b.html>.
 - [12] F. Tramer, N. Carlini, W. Brendel, and A. Madry. On Adaptive Attacks to Adversarial Example Defenses. NeurIPS, 2020. <https://arxiv.org/abs/2002.08347>.
 - [13] F. Croce et al. RobustBench: A Standardized Adversarial Robustness Benchmark. NeurIPS Datasets and Benchmarks, 2021. <https://robustbench.github.io/>.